

Frontend Technical Debt

Overview

The Android client already contains partial refactors in the right direction, but debt is still concentrated in app bootstrap, large feature owners, and duplicated cross-cutting logic.

High-value debt areas

App bootstrap

- `MyApplication` initializes locale, ads, Firebase, OneSignal, Cloudinary, and notification behavior in one place
- the file also still contains source-coded third-party configuration values

Feed player surface

- `YenkasaPlayerView` mixes rendering, playback, wallet state, search, community chrome, monetization hooks, and overlay animation
- difficult to test in smaller slices

Chat lifecycle surface

- `ChatActivity` is improved by controller extraction, but lifecycle and interaction volume remain high
- push suppression, realtime events, sounds, attachments, and calls still converge here

Livestream room surface

- `LiveStreamActivity` mixes Agora session management, socket protocol handling, room state, comments, gifts, and host controls

State storage

- `TokenManager` stores auth, identity, permissions, push ids, selected communities, and feed cache
- this makes local state evolution harder to reason about

Known operational debt

- BODY-level HTTP logging in the shared API client
- notification routing logic duplicated across deep-link and type-based interpretation
- role/capability visibility logic duplicated between client and server concepts

Recommended future improvements

1. move secrets to safer delivery mechanisms
2. continue feature-by-feature coordinator extraction
3. split local persistence by domain instead of one broad manager
4. define typed contracts for intents, notifications, and socket events
5. add more automated regression coverage around feed, chat, and livestream flows